Introduction to Theoretical Computer Science

CS2490 - Session 1: What is this course about?

Pierre Cagne

Fall 2026

August 17, 2026

Computer Science

Computer Science

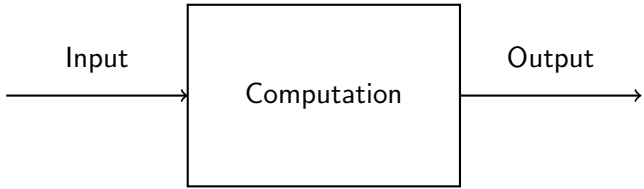
Computer Science

is the study of *computations*

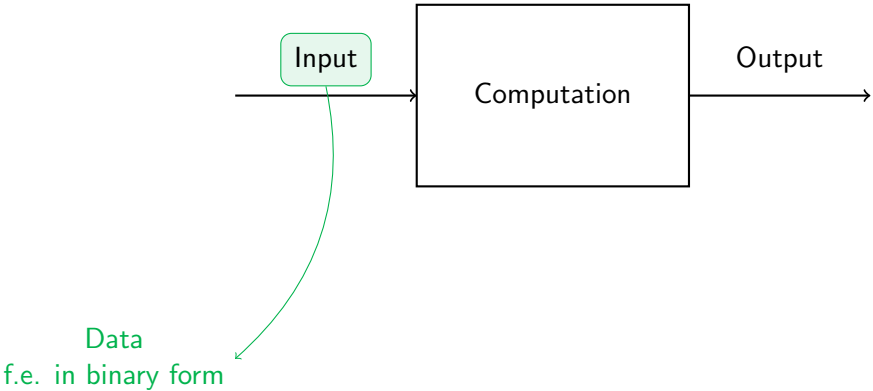
Computer Science

is the mathematical study of *computations*

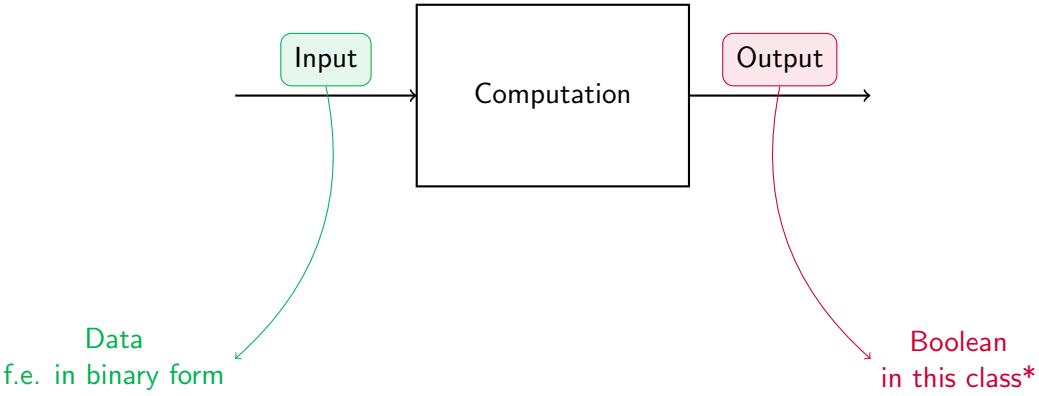
Abstract Computation



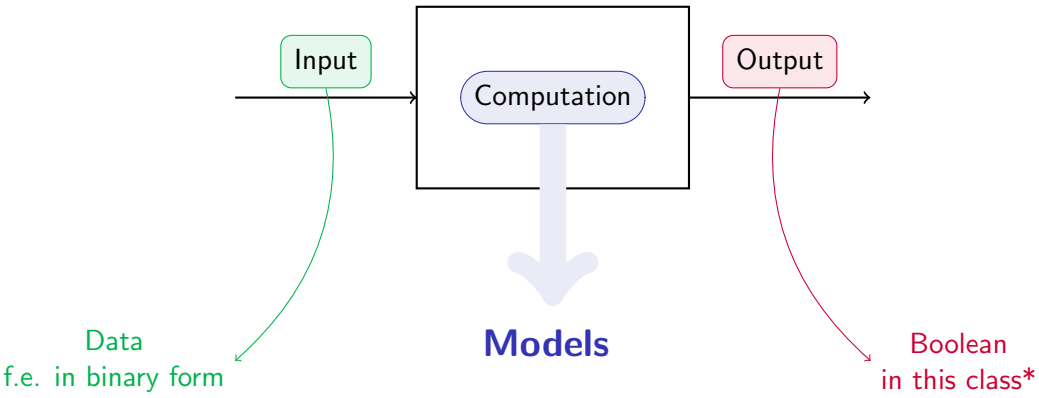
Abstract Computation



Abstract Computation



Abstract Computation



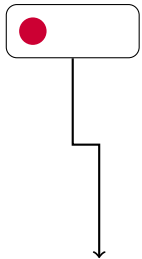
Finite State Machines

All the models we will study in this course are *finite state machines*.

Think of FSM as a black box with:

- colored LEDs on the top that can light up one at a time,
- internal wiring that dictates how the LEDs change based on the input.

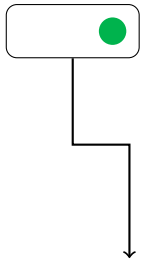
Example



Rule: in state , reading 1
change to state

1101100111010110001

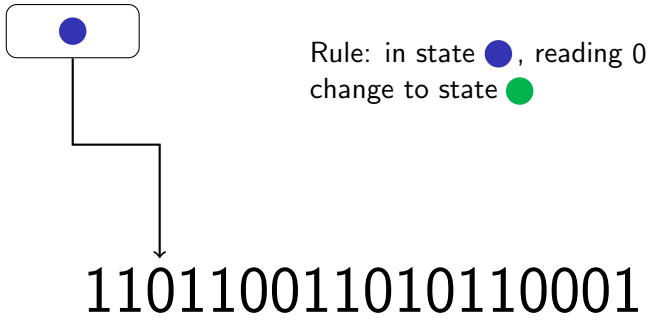
Example



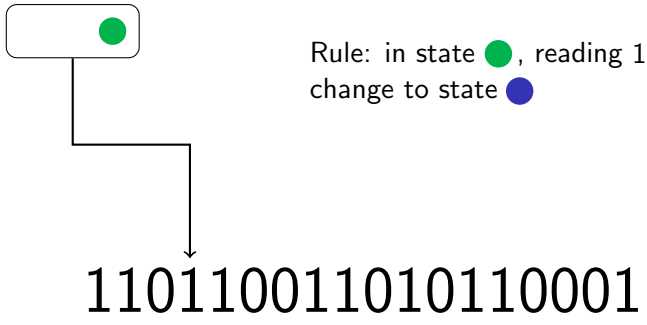
Rule: in state , reading 1
change to state

1101100111010110001

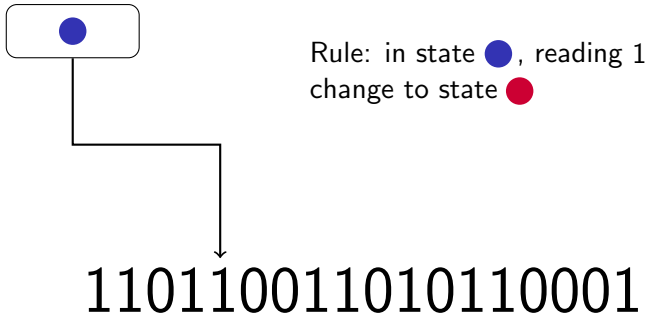
Example


Rule: in state , reading 0
change to state
1101100111010110001

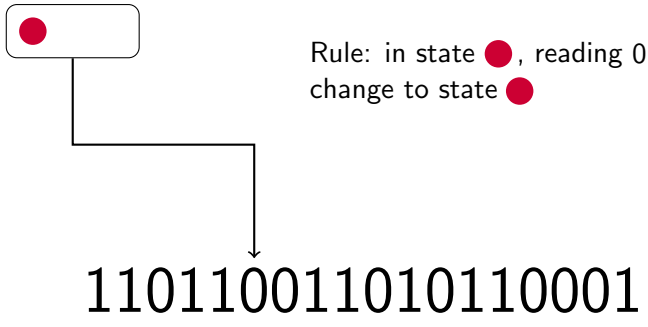
Example


Rule: in state , reading 1
change to state
1101100111010110001

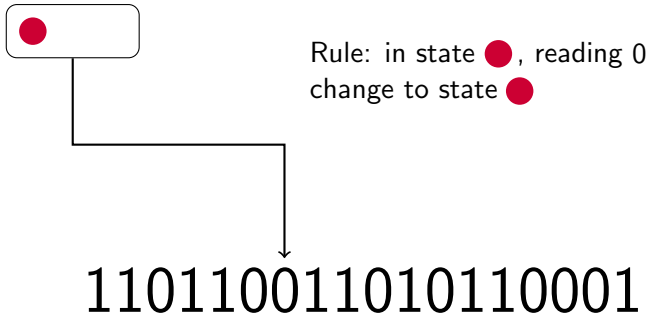
Example


Rule: in state , reading 1
change to state
1101100111010110001

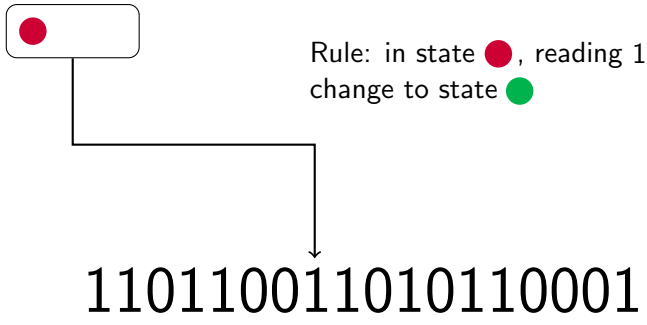
Example


Rule: in state , reading 0
change to state
1101100111010110001

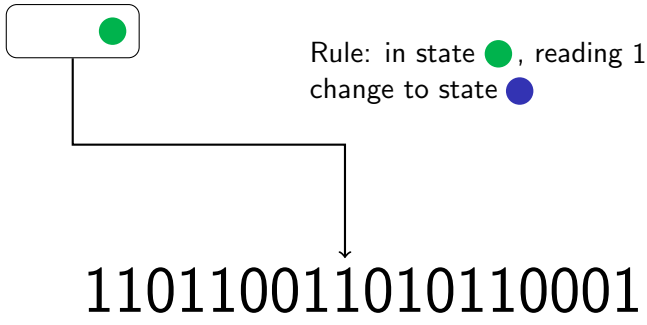
Example


Rule: in state , reading 0
change to state
1101100111010110001

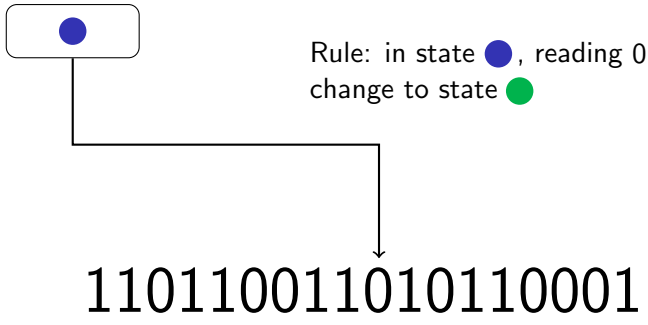
Example


Rule: in state , reading 1
change to state
1101100111010110001

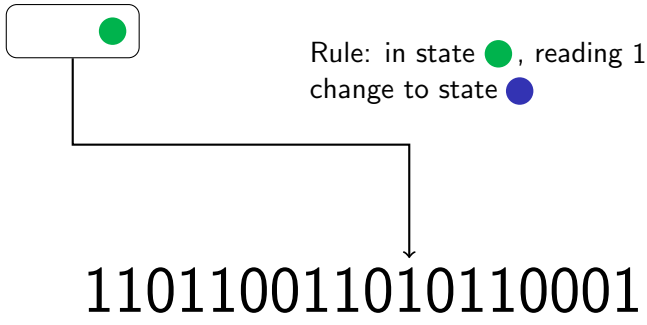
Example


Rule: in state , reading 1
change to state
1101100111010110001

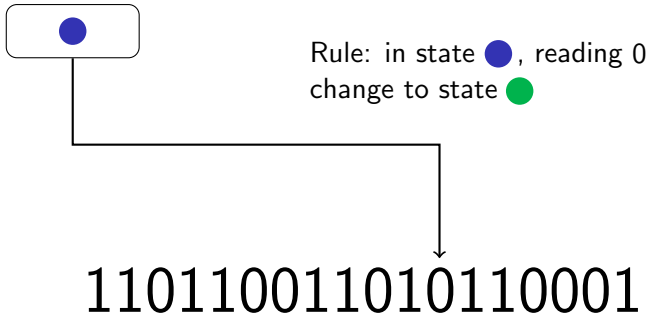
Example


Rule: in state , reading 0
change to state
1101100110101100 01

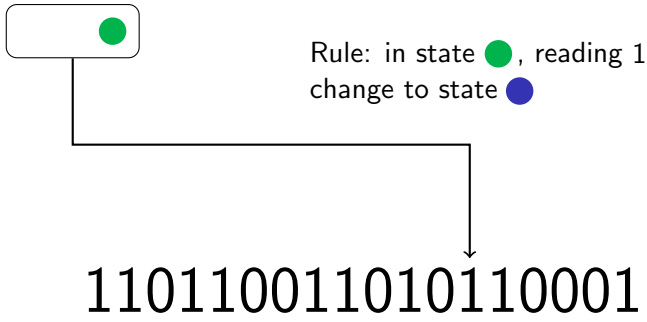
Example


Rule: in state , reading 1
change to state
1101100110101100 01

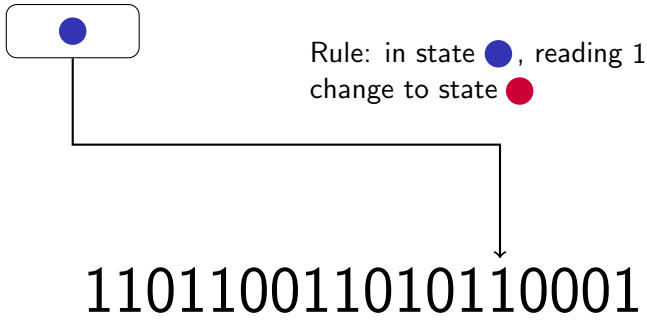
Example


Rule: in state , reading 0
change to state
1101100110101100 01

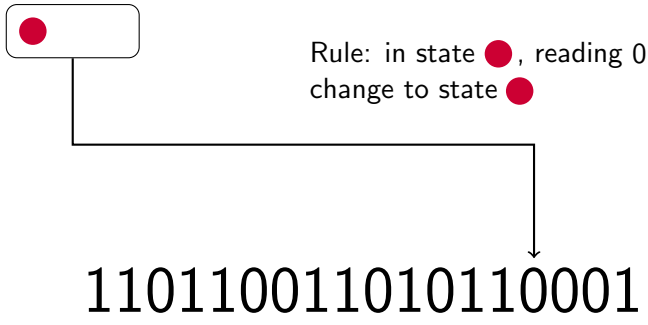
Example


Rule: in state , reading 1
change to state
1101100110101100001

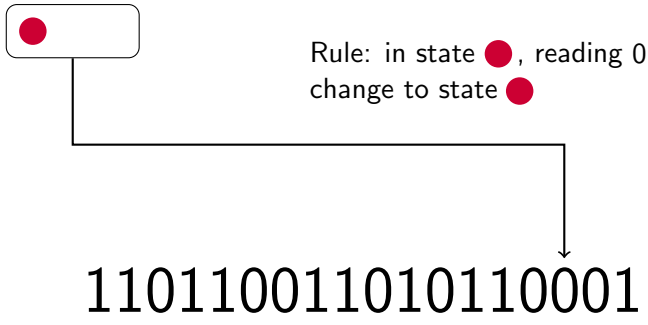
Example


Rule: in state , reading 1
change to state
1101100110101100 01

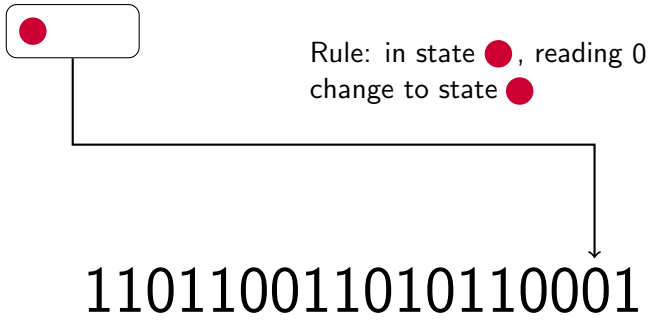
Example


Rule: in state , reading 0
change to state
1101100111010110001

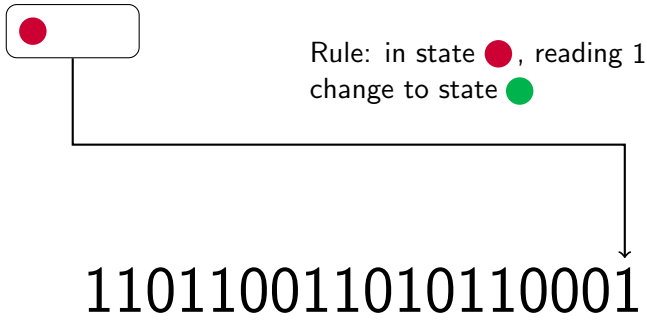
Example


Rule: in state , reading 0
change to state
1101100110101100 01

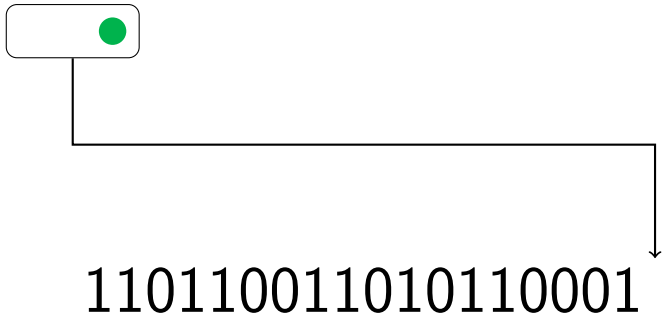
Example


Rule: in state , reading 0
change to state
1101100111010110001

Example


Rule: in state , reading 1
change to state
1101100110101100 01

Example



A less random example

How to count the parity of 1s in a binary input?

A less random example

How to count the parity of 1s in a binary input?

Make your states (the LEDs) mirror meaningful features of your input:

A less random example

How to count the parity of 1s in a binary input?

Make your states (the LEDs) mirror meaningful features of your input:

-  lights up for an even number of 1s,

A less random example

How to count the parity of 1s in a binary input?

Make your states (the LEDs) mirror meaningful features of your input:

-  lights up for an even number of 1s,
-  lights up for an odd number of 1s.

A less random example

How to count the parity of 1s in a binary input?

Make your states (the LEDs) mirror meaningful features of your input:

-  lights up for an even number of 1s,
-  lights up for an odd number of 1s.

Make the rules (the wiring of the LEDs) connect these features:

A less random example

How to count the parity of 1s in a binary input?

Make your states (the LEDs) mirror meaningful features of your input:

- ● lights up for an even number of 1s,
- ● lights up for an odd number of 1s.

Make the rules (the wiring of the LEDs) connect these features:

- In state ●, reading 1, change to ●,

A less random example

How to count the parity of 1s in a binary input?

Make your states (the LEDs) mirror meaningful features of your input:

-  lights up for an even number of 1s,
-  lights up for an odd number of 1s.

Make the rules (the wiring of the LEDs) connect these features:

- In state  , reading 1, change to  ,
- In state  , reading 1, change to  ,

A less random example

How to count the parity of 1s in a binary input?

Make your states (the LEDs) mirror meaningful features of your input:

-  lights up for an even number of 1s,
-  lights up for an odd number of 1s.

Make the rules (the wiring of the LEDs) connect these features:

- In state  , reading 1, change to  ,
- In state  , reading 1, change to  ,
- In state  , reading 0, stay in  ,

A less random example

How to count the parity of 1s in a binary input?

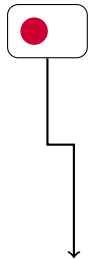
Make your states (the LEDs) mirror meaningful features of your input:

- red lights up for an even number of 1s,
- green lights up for an odd number of 1s.

Make the rules (the wiring of the LEDs) connect these features:

- In state red, reading 1, change to green,
- In state green, reading 1, change to red,
- In state red, reading 0, stay in red,
- In state green, reading 0, stay in green.

Example



Rule: in state  , reading 1
change to state 

1101100110101100 01

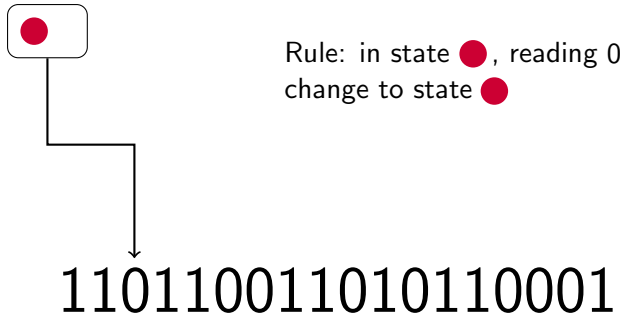
Example



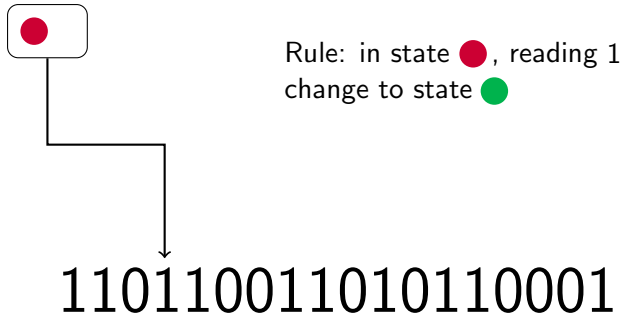
Rule: in state  , reading 1
change to state 

1101100110101100 01

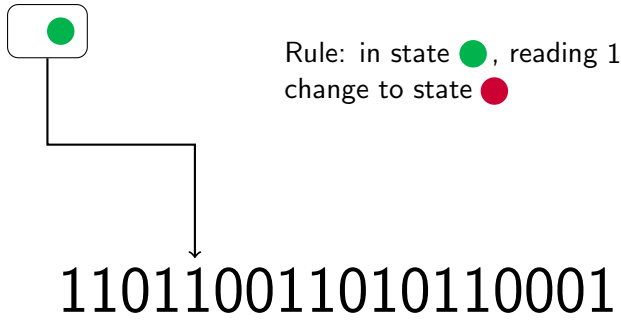
Example


Rule: in state , reading 0
change to state
1101100110101101 0001

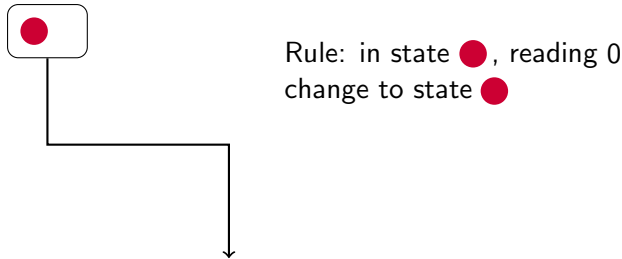
Example


Rule: in state , reading 1
change to state
1101100110101101 0001

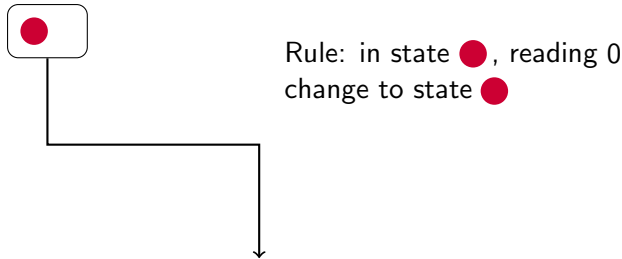
Example


Rule: in state , reading 1
change to state
1101100110101101 0001

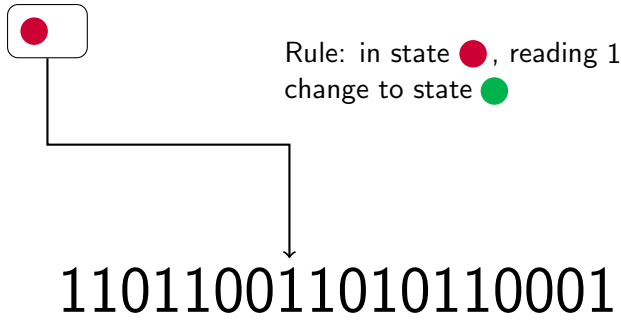
Example


Rule: in state , reading 0
change to state
1101100110101100 01

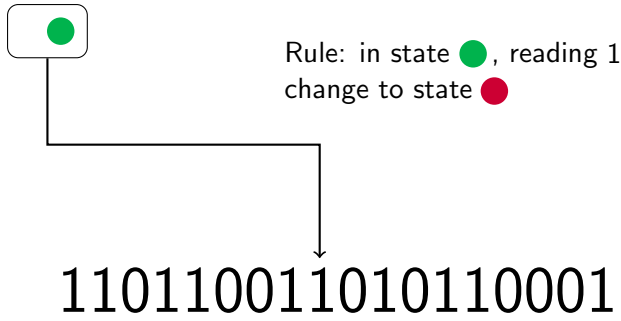
Example


Rule: in state , reading 0
change to state
1101100111010110001

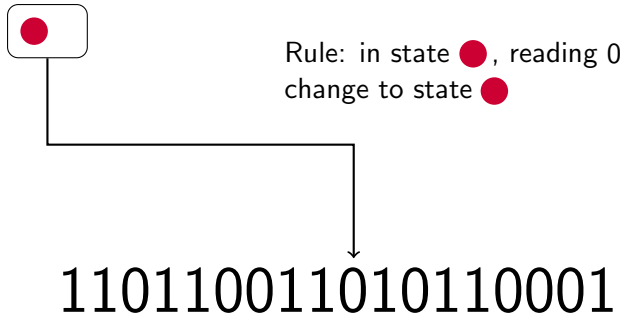
Example


Rule: in state , reading 1
change to state
1101100111010110001

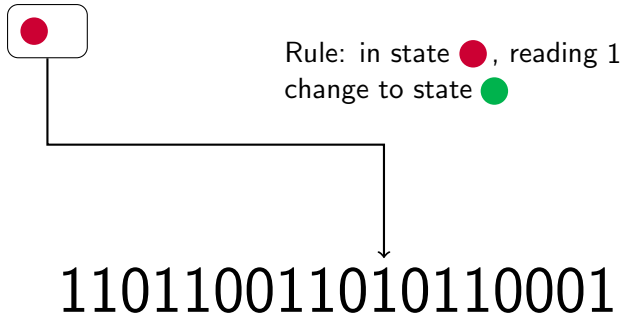
Example


Rule: in state , reading 1
change to state
1101100111010110001

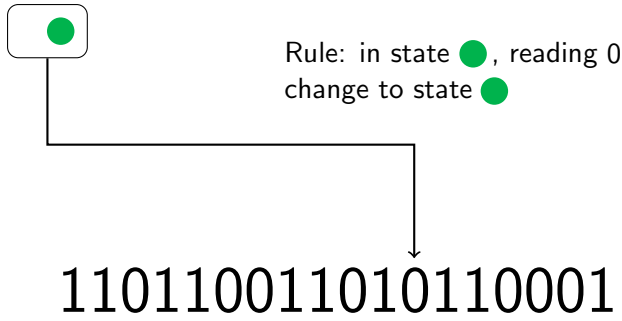
Example


Rule: in state , reading 0
change to state
1101100110101100 01

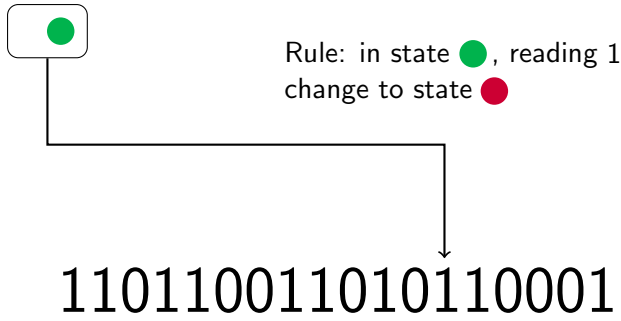
Example


Rule: in state , reading 1
change to state
1101100110101100 01

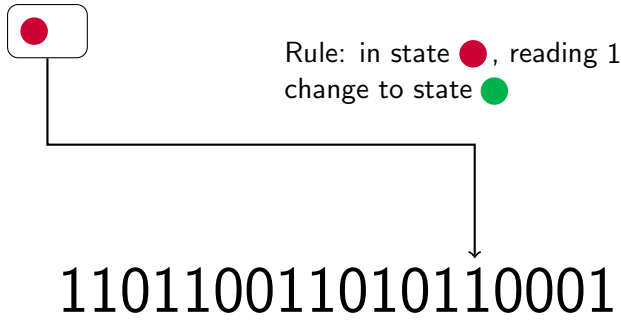
Example


Rule: in state , reading 0
change to state
1101100110101100 01

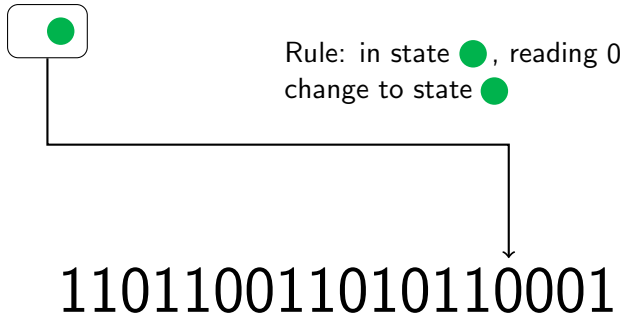
Example


Rule: in state , reading 1
change to state
1101100110101100 01

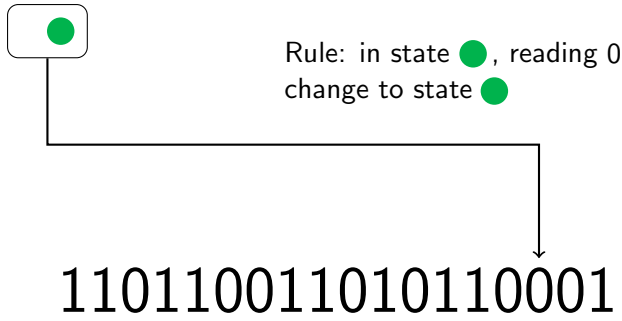
Example


Rule: in state , reading 1
change to state
1101100110101100 01

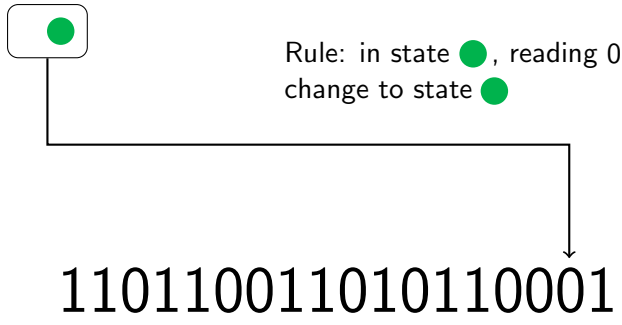
Example


Rule: in state , reading 0
change to state
1101100110101100 01

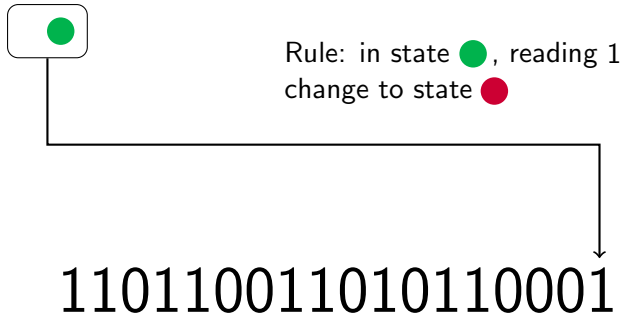
Example


Rule: in state , reading 0
change to state
1101100110101100001

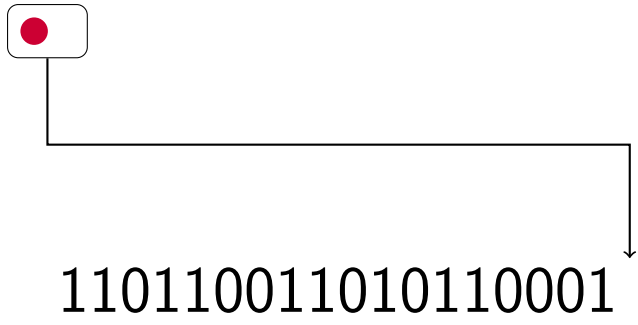
Example


Rule: in state , reading 0
change to state
11011001101011000 1

Example


Rule: in state , reading 1
change to state
1101100110101100001

Example



The goal of this semester

Make that intuitive notion of an FSM with LEDs and wiring precise and formal, so that one can reason about its behavior and properties.

We will see FSM that can:

The goal of this semester

Make that intuitive notion of an FSM with LEDs and wiring precise and formal, so that one can reason about its behavior and properties.

We will see FSM that can:

1. only read once from left to right, one letter at a time: finite automata (DFA/NFA),

The goal of this semester

Make that intuitive notion of an FSM with LEDs and wiring precise and formal, so that one can reason about its behavior and properties.

We will see FSM that can:

1. only read once from left to right, one letter at a time: finite automata (DFA/NFA),
2. only read once from left to right, one letter at a time, but can remember some information: pushdown automata (PDA),

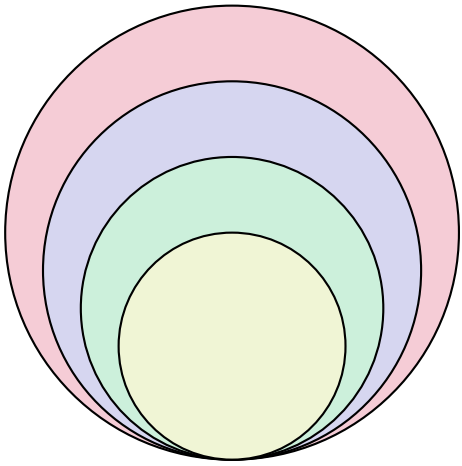
The goal of this semester

Make that intuitive notion of an FSM with LEDs and wiring precise and formal, so that one can reason about its behavior and properties.

We will see FSM that can:

1. only read once from left to right, one letter at a time: finite automata (DFA/NFA),
2. only read once from left to right, one letter at a time, but can remember some information: pushdown automata (PDA),
3. go back and forth on the input, and can even change it: Turing machines (TM),

Chomsky Hierarchy



Not computable at all

Need Turing Machines to compute them

Need Pushdown Automata to compute them

Computable by Finite Automata